

Software Engineer — Step 4

Software Projects

Roadmap objective: Design, build, test and document maintainable software

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Requirements
- Architecture
- Modules
- APIs
- Persistence
- Testing
- Logging
- Git
- Documentation
- Deployment

Core concepts

1. Start with requirements and acceptance criteria before coding.
2. Break large systems into modules with clear responsibilities and interfaces.
3. Tests should target important behavior and edge cases; not every line needs a test.
4. A maintainable project is understandable to another developer without a live explanation from its author.

Concept map

```
Software Projects
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Requirements

Study requirements through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Architecture

Study architecture through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Modules

Study modules through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. APIs

Study apis through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Persistence

Study persistence through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Testing

Study testing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Project structure

```
src/  
  config/  
  routes/  
  controllers/  
  services/  
  repositories/  
  models/  
  tests/  
  app.js
```

The goal is not a magic folder structure.
The goal is clear responsibilities and low coupling.

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Unit test	Small behavior	Fast
Integration test	Components together	Broader confidence
E2E test	User flow/system	Higher cost

Common mistakes

- No acceptance criteria
- God classes/modules
- Tests that only test implementation details
- No logging/error strategy
- README that cannot reproduce setup

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Write requirements before code.
- Write down what you expected, what happened, and why.

Lab 2

- Create a module boundary diagram.
- Write down what you expected, what happened, and why.

Lab 3

- Implement one feature.
- Write down what you expected, what happened, and why.

Lab 4

- Add unit and integration coverage where valuable.
- Write down what you expected, what happened, and why.

Lab 5

- Add structured error/logging behavior.
- Write down what you expected, what happened, and why.

Lab 6

- Ask another person to run the project using only the README.
- Write down what you expected, what happened, and why.

Mini-project

Build a maintainable application with requirements, modular architecture, tests, logging, Git workflow and README. Ask another person to set it up using only the README.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What is modularity?

- Answer structure: definition → example → trade-off/limitation → practical use.

Unit vs integration test?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why write requirements first?

- Answer structure: definition → example → trade-off/limitation → practical use.

What makes code maintainable?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why is logging useful?

- Answer structure: definition → example → trade-off/limitation → practical use.

What should a README guarantee?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Requirement	Write this in your own words.	Give one project example.
Module	Write this in your own words.	Give one project example.
Coupling	Write this in your own words.	Give one project example.
Cohesion	Write this in your own words.	Give one project example.
Unit test	Write this in your own words.	Give one project example.
Integration test	Write this in your own words.	Give one project example.
Logging	Write this in your own words.	Give one project example.
Architecture	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Refactor	Write this in your own words.	Give one project example.
Documentation	Write this in your own words.	Give one project example.

Self-test

- Explain Requirements without using its textbook definition.
- Show a small example of Architecture.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Requirements.
- I can explain and demonstrate Architecture.
- I can explain and demonstrate Modules.
- I can explain and demonstrate APIs.
- I can explain and demonstrate Persistence.
- I can explain and demonstrate Testing.
- I can explain and demonstrate Logging.
- I can explain and demonstrate Git.
- I can explain and demonstrate Documentation.
- I can explain and demonstrate Deployment.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://docs.github.com/>
- <https://git-scm.com/doc>

Recommended videos

- Building Software Projects (Existing DB resource 119)
- Full Stack Project Tutorial (Existing DB resource 115)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.